

Module: tf.compat.v1.raw_ops Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/raw_ops

Public API for tf._api.v2.raw_ops namespace

Function Signatures

```
Abort(...)
Abs(...)
AccumulateNV2(...)
AccumulatorApplyGradient(...)
AccumulatorNumAccumulated(...)
```

Documentation

Public API for tf._api.v2.raw_ops namespace

Functions

`Abort(...)`: Raise a exception to abort the process when called.

`Abs(...)`: Computes the absolute value of a tensor.

AccumulateNV2(...): Returns the element-wise sum of a list of tensors.

AccumulatorApplyGradient(...): Applies a gradient to a given accumulator.

AccumulatorNumAccumulated(...): Returns the number of gradients aggregated in the given accumulators.

AccumulatorSetGlobalStep(...): Updates the accumulator with a new value for `global_step`.

AccumulatorTakeGradient(...): Extracts the average gradient in the given `ConditionalAccumulator`.

Acos(...): Computes acos of x element-wise.

Acosh(...): Computes inverse hyperbolic cosine of x element-wise.

Add(...): Returns $x + y$ element-wise.

AddManySparseToTensorsMap(...): Add an N-minibatch `SparseTensor` to a `SparseTensorsMap`, return N handles.

AddN(...): Add all input tensors element wise.

AddSparseToTensorsMap(...): Add a `SparseTensor` to a `SparseTensorsMap` return its handle.

AddV2(...): Returns $x + y$ element-wise.

AdjustContrast(...): Deprecated. Disallowed in `GraphDef` version ≥ 2 .

AdjustContrastv2(...): Adjust the contrast of one or more images.

AdjustHue(...): Adjust the hue of one or more images.

AdjustSaturation(...): Adjust the saturation of one or more images.

All(...): Computes the "logical and" of elements across dimensions of a tensor.

AllCandidateSampler(...): Generates labels for candidate sampling with a learned

unigram distribution.

AllToAll(...): An Op to exchange data across TPU replicas.

Angle(...): Returns the argument of a complex number.

AnonymousHashTable(...): Creates a uninitialized anonymous hash table.

AnonymousIterator(...): A container for an iterator resource.

AnonymousIteratorV2(...): A container for an iterator resource.

AnonymousIteratorV3(...): A container for an iterator resource.

AnonymousMemoryCache(...)

AnonymousMultiDeviceIterator(...): A container for a multi device iterator resource.

AnonymousMultiDeviceIteratorV3(...): A container for a multi device iterator resource.

AnonymousMutableDenseHashTable(...): Creates an empty anonymous mutable hash table that uses tensors as the backing store.

AnonymousMutableHashTable(...): Creates an empty anonymous mutable hash table.

AnonymousMutableHashTableOfTensors(...): Creates an empty anonymous mutable hash table of vector values.

AnonymousRandomSeedGenerator(...)

AnonymousSeedGenerator(...)

Any(...): Computes the "logical or" of elements across dimensions of a tensor.

ApplyAdaMax(...): Update '*var' according to the AdaMax algorithm.

ApplyAdadelta(...): Update '*var' according to the adadelta scheme.

ApplyAdagrad(...): Update '*var' according to the adagrad scheme.

ApplyAdagradDA(...): Update '*var' according to the proximal adagrad scheme.

ApplyAdagradV2(...): Update '*var' according to the adagrad scheme.

ApplyAdam(...): Update '*var' according to the Adam algorithm.

ApplyAddSign(...): Update '*var' according to the AddSign update.

ApplyCenteredRMSPProp(...): Update '*var' according to the centered RMSPProp algorithm.

ApplyFtrl(...): Update '*var' according to the Ftrl-proximal scheme.

ApplyFtrlV2(...): Update '*var' according to the Ftrl-proximal scheme.

ApplyGradientDescent(...): Update '*var' by subtracting 'alpha' * 'delta' from it.

ApplyMomentum(...): Update '*var' according to the momentum scheme.

ApplyPowerSign(...): Update '*var' according to the AddSign update.

ApplyProximalAdagrad(...): Update 'var' and 'accum' according to FOBOS with Adagrad learning rate.

ApplyProximalGradientDescent(...): Update '*var' as FOBOS algorithm with fixed learning rate.

ApplyRMSPProp(...): Update '*var' according to the RMSPProp algorithm.

ApproxTopK(...): Returns min/max k values and their indices of the input operand in an approximate manner.

ApproximateEqual(...): Returns the truth value of $\text{abs}(x-y) < \text{tolerance}$ element-wise.

ArgMax(...): Returns the index with the largest value across dimensions of a tensor.

ArgMin(...): Returns the index with the smallest value across dimensions of a tensor.

AsString(...): Converts each entry in the given tensor to strings.

Asin(...): Computes the trigonometric inverse sine of x element-wise.

Asinh(...): Computes inverse hyperbolic sine of x element-wise.

Assert(...): Asserts that the given condition is true.

AssertCardinalityDataset(...)

AssertNextDataset(...): A transformation that asserts which transformations happen next.

AssertPrevDataset(...): A transformation that asserts which transformations happened previously.

Assign(...): Update 'ref' by assigning 'value' to it.

AssignAdd(...): Update 'ref' by adding 'value' to it.

AssignAddVariableOp(...): Adds a value to the current value of a variable.

AssignSub(...): Update 'ref' by subtracting 'value' from it.

AssignSubVariableOp(...): Subtracts a value from the current value of a variable.

AssignVariableOp(...): Assigns a new value to a variable.

AssignVariableXlaConcatND(...): Concatenates input tensor across all dimensions.

Atan(...): Computes the trigonometric inverse tangent of x element-wise.

Atan2(...): Computes arctangent of y/x element-wise, respecting signs of the arguments.

Atanh(...): Computes inverse hyperbolic tangent of x element-wise.

AudioSpectrogram(...): Produces a visualization of audio data over time.

`AudioSummary(...)`: Outputs a Summary protocol buffer with audio.

`AudioSummaryV2(...)`: Outputs a Summary protocol buffer with audio.

`AutoShardDataset(...)`: Creates a dataset that shards the input dataset.

`AvgPool(...)`: Performs average pooling on the input.

`AvgPool3D(...)`: Performs 3D average pooling on the input.

`AvgPool3DGrad(...)`: Computes gradients of average pooling function.

`AvgPoolGrad(...)`: Computes gradients of the average pooling function.

`BandedTriangularSolve(...)`

`Barrier(...)`: Defines a barrier that persists across different graph executions.

`BarrierClose(...)`: Closes the given barrier.

`BarrierIncompleteSize(...)`: Computes th